

Notation $D = \{(x^{(i)}, y^{(i)})\}_{i=1}^n, x^{(i)} \in \mathbb{R}^d, \hat{y}^{(i)} = h(x^{(i)})$. n examples, d features.

Linear/logistic: $h_w(x) = w^\top x = \sum_{j=0}^d w_j x_j$ with dummy $x_0 = 1$, so bias is w_0 ; logistic $h_w(x) = \sigma(w^\top x)$. Design matrix $X \in \mathbb{R}^{n \times (d+1)}$ (row per example, leading 1); targets $Y \in \mathbb{R}^n$. $\text{MSE } J_{MSE}(w) = \frac{1}{2n} \sum_i (h_w(x^{(i)}) - y^{(i)})^2$. **DT**: entropy $H(Y) = -\sum_k P(y_k) \log_2 P(y_k)$; conditional $H(Y|X) = \sum_x P(X=x) H(Y|X=x)$; gain $IG(Y; X) = H(Y) - H(Y|X)$.

Reg/Kernel: $J_{reg}(w) = J(w) + \lambda R(w)$, $\lambda \geq 0$; $\|w\|_p = (\sum_j |w_j|^p)^{1/p}$. Dual $h_\alpha(x) = \sum_j \alpha_j \langle x^{(j)}, x \rangle$; kernel $k_\phi(u, v) = \langle \phi(u), \phi(v) \rangle$; hyperplane $h_w(x) = \text{sign}(w^\top x)$, distance $|w^\top x|/\|w\|$.

NN: L layers (excludes input). Layer l : $W^{[l]}$, bias $b^{[l]}$, activation $g^{[l]}$; $\hat{y} = g^{[L]}((W^{[L]})^\top g^{[L-1]}(\dots g^{[1]}((W^{[1]})^\top x)))$. Entry $W_{ij}^{[l]}$ connects source i in layer $l-1$ to destination j in layer l (so $W_{ij}^{[l]} \in \mathbb{R}^{(\text{in}) \times (\text{out})}$).

CNN: * convolution; input $I_H \times I_W \times C$, kernel $K \times K$, stride S , padding P . Output side $1 + \lfloor (I_W + 2P - K)/S \rfloor$. Each kernel matches input channels; #kernels = #output feature maps.

RNN: $h^{[t]} = g((W^{[xh]})^\top x^{[t]} + (W^{[hh]})^\top h^{[t-1]} + b)$, $h^{[0]} = \mathbf{0}$, T time steps; output $\hat{y}^{[t]} = \sigma((W^{[hy]})^\top h^{[t]})$. Bidirectional $h^{[t]} = [\vec{h}^{[t]}, \overleftarrow{h}^{[t]}]$; stacked $h^{[l,t]} = f^{[l]}(h^{[l-1,t]}, h^{[l,t-1]})$.

Attention: $Q = W^q X$, $K = W^k X$, $V = W^v X$; $C = V \text{softmax}_{\text{col}}(K^\top Q / \sqrt{d_k}) \in \mathbb{R}^{d_v \times T}$. Position: $x^{[t]} \leftarrow x^{[t]} + PE^{[t]}, PE_{2k}^{[t]} = \sin(t/L^{2k/d}), PE_{2k+1}^{[t]} = \cos(t/L^{2k/d})$.

Identities **Chain**: $\frac{d}{dx} f(g(x)) = f'(g(x)) g'(x)$; multivariate $\nabla_w J(h(w)) = h'(w) J'(h)$.

Gradients: $\nabla_w(w^\top x) = x$, $\nabla_w(w^\top w) = 2w$, $\nabla_w(w^\top Aw) = (A + A^\top)w$, $\nabla_w \|Xw - Y\|^2 = 2X^\top(Xw - Y)$.

MSE gradient: $\partial J_{MSE} / \partial w_j = \frac{1}{n} \sum_i (w^\top x^{(i)} - y^{(i)}) x_j^{(i)}$; ridge adds $+\lambda w_j$.

Normal equation: $w = (X^\top X)^{-1} X^\top Y$; ridge $w = (X^\top X + \lambda \mathbb{I})^{-1} X^\top Y$.

Activations: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$; $\tanh'(z) = 1 - \tanh^2(z)$; $\text{ReLU}'(z) = \mathbb{I}[z > 0]$.

BCE+sigmoid: if $\hat{y} = \sigma(z)$ and $\ell = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$, then $\partial \ell / \partial z = \hat{y} - y$.

Softmax: $s_i = e^{z_i} / \sum_k e^{z_k}$, $\partial s_i / \partial z_j = s_i(\delta_{ij} - s_j)$; CE+softmax $\partial \ell / \partial z_j = \hat{y}_j - y_j$.

Log-sum-exp: $\log \sum_i e^{z_i} = z^* + \log \sum_i e^{z_i - z^*}$, $z^* = \max_i z_i$.

Common entropies: $H(1) = 0$, $H(\frac{1}{2}, \frac{1}{2}) = 1$, $H(\frac{1}{3}, \frac{2}{3}) \approx 0.918$, $H(\frac{3}{5}, \frac{2}{5}) \approx 0.971$.

1 Search & Adversarial Search

PEAS (Performance / Environment / Actuators / Sensors). **Problem** = (states, init, actions, transition, goal-test, cost $c(s, a, s')$); tree vs. graph (+ visited).

Uninformed **BFS / UCS / IDS** complete & optimal (BFS / IDS unit-cost; UCS needs $c \geq \epsilon > 0$, $C^* < \infty$; UCS = Dijkstra). **DFS**: $O(b^m)$

time, $O(bm)$ space, **not optimal, complete only with visited + finite**.

Informed (A*) **A***: $f=g+h$, $h(\text{goal})=0$. **Admissible**: $h \leq h^*$. **Consistent**: $h(n) \leq c(n, a, n') + h(n')$ (triangle); consistent $\Rightarrow$ admissible. Optimality (Hart '68): tree if h admissible; graph (v-sided) if h consistent. **Dominance**: $h_1 \geq h_2$ (admissible) $\Rightarrow h_1$ expands fewer. **Relaxed problem**: optimal cost gives admissible h . **Greedy ($f=h$) not optimal**.

Local Incomplete, suboptimal, any-time. **Hill-climb**: best-evaluation neighbour; stuck at **local max / shoulder / ridge**. Escapes:

- Random restart.
- Simulated annealing**: accept worse with $P=e^{-\Delta/T}$.
- Beam search.
- Genetic algorithm.

Adversarial 2-player zero-sum turn-taking; A maximises, B minimises. **Minimax**: $V(s) = u(s)$ at terminal, else $\max_{s'} V(s')$ or $\min_{s'} V(s')$, $O(b^m)$ time / $O(bm)$ space, optimal vs. optimal opponent. **α - β pruning**: α / β best for A / B; max-prune $v \geq \beta$, min-prune $v \leq \alpha$; **perfect order** $O(b^{m/2})$. **Cutoff + eval**: is_cutoff returns eval(s) between min / max utility; **admissibility / consistency don't apply**.

2 Classical Machine Learning
Supervised: labelled D , learn $h : X \rightarrow Y$. **Regression** ($y \in \mathbb{R}$, MSE / MAE); **Classification** ($y \in \{1, \dots, K\}$; accuracy, $P = \frac{TP}{TP+FP}$, $R = \frac{TP}{TP+FN}$, $F1 = \frac{2}{1/P+1/R}$). Train / test split; performance on test = generalisation (assumes shared ground truth).

2.1 Decision Trees

DT: internal nodes = attribute tests, branches = outcomes, leaves = class; predict via nested if-else. Inputs categorical $x \in \{c_1, \dots, c_m\}^d$, target $\in \{1, \dots, K\}$. Representational completeness: any consistent finite-categorical set has a perfectly-classifying DT. **Optimal (smallest) DT is NP-hard**. Greedy **ID3**: at each node pick $a^* = \arg \max_a IG(Y; a)$, partition rows by a^* 's values, recurse.

2.2 Linear Regression

Entropy / IG $H(Y|X) = \sum_x P(X=x) H(Y|X=x)$ (weighted average of specific entropies); IG = uncertainty reduction after observing X . Common values in miscMath. **Overfit / pruning** DT fits noise $\Rightarrow$ overfit. **Pre-pruning**:

- Max-depth**: longest root-leaf path.
- Min-leaf-size**: refuse splits making a leaf below threshold $\rightarrow$ majority.

Base cases

- Rows empty $\rightarrow$ default (parent's majority).
- Pure $\rightarrow$ that label.
- No attributes left $\rightarrow$ majority(rows) (handles noise / inconsistent data).

Real-world: continuous $\rightarrow$ binned intervals; missing $\rightarrow$ most-common (overall / per-class), sample by prior, drop attribute / row.

Overfit / pruning DT fits noise $\Rightarrow$ overfit. **Pre-pruning**:

- Max-depth**: longest root-leaf path.
- Min-leaf-size**: refuse splits making a leaf below threshold $\rightarrow$ majority.

Model $h_w(x)=w^\top x$, J_{MSE} in miscMath ($\frac{1}{2}$ for clean derivative).

Normal equation
$$w = (X^\top X)^{-1} X^\top Y$$

(set $X^\top(Xw - Y) = 0$; closed form, no γ , but $O(d^3)$ inversion, needs $X^\top X$ invertible (linearly independent features); fails for non-linear models.

Gradient descent

$$w_j \leftarrow w_j - \gamma \partial J / \partial w_j$$

Learning rate γ : **too large diverges, too small slow**. Compute all $\partial J / \partial w_j$ from current w , then update simultaneously. Variants:

- Batch**: all n examples per step.
- Mini-batch**: subset; cheaper, noisy gradient escapes plateaus.
- SGD**: one example per step.

Convexity Convex $\Rightarrow$ every local min global; strictly convex $\Rightarrow$ unique global min. MSE + linear is convex; if features linearly independent then strictly convex $\Rightarrow$ GD finds unique global min.

Feature scaling (for GD)

- Min-max**: $x' = (x - \min) / (\max - \min)$.
- Standardise**: $x' = (x - \mu) / \sigma$.

Equalises curvature $\rightarrow$ faster convergence; **not needed for normal equation**.

Feature transform $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^M$. **Polynomial** ϕ_{pk} degree k ($\Rightarrow$ **Polynomial regression**), log, exp. Linear-in- w after $\phi \Rightarrow$ same algorithms.

2.3 Logistic Regression

Sigmoid $\sigma(z) = 1 / (1 + e^{-z})$, $\sigma' = \sigma(1 - \sigma)$. **Model** $h_w(x) = \sigma(w^\top x) = P(y=1|x)$ (confidence); predict 1 iff $h_w(x) \geq \tau$ (**Decision threshold**, default .5). Decision boundary: $w^\top x = 0$ (hyperplane).

BCE loss $\ell = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$, $J_{BCE} = \frac{1}{n} \sum_i \ell$. Logistic regression = sigmoid model + BCE. **MSE here is non-convex** (sigmoid keeps S-shape under squaring); BCE (i.e. $-\log \sigma$) is convex (preserved under feature transforms) $\Rightarrow$ GD finds global min. No closed form (transcendental). Gradient:

$$\partial J / \partial w_j = \frac{1}{n} \sum_i (h_w(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Same form as linear regression. Non-linearly separable $\rightarrow$ feature transforms.

Multi-class (K) Reduce to binary:

- One-vs-Rest**: K classifiers (class- k vs. rest); predict $\arg \max_k h_{w[k]}$.
- One-vs-One**: $K(K-1)/2$ pairwise; majority vote.

Multi-label ($y \in \{0, 1\}^K$):

- Binary relevance**: one classifier per label; **ignores correlations**.
- Classifier chain**: feed prior predictions as features.
- Label powerset**: each label-set = one multi-class label.

2.4 Bias-Variance & Tuning

High bias / underfit: simple model on complex truth; train & test both high; more data won't help. **High variance / overfit**: complex model on scarce / noisy data; train $\ll$ test; retraining on a fresh sample gives a very different model. Diagnose via train-vs-test gap. Generalisation depends on data (relevance, noise, balance, quantity) and model complexity.

Hyperparameters Set pre-training (γ , polynomial degree, batch size, λ); tune on a **validation set**. Methods:

- Grid search (exhaustive).
- Random search.
- Local search (hill-climb).
- Successive halving.
- Bayesian optimisation.

2.5 Regularization

Overfit often correlates with large-magnitude w . Regularised loss in miscMath. Variants:

- Ridge** (L_2): $R = \frac{\lambda}{2} \|w\|_2^2$; gradient adds λw_j . Closed form $w = (X^\top X + \lambda \mathbb{I})^{-1} X^\top Y$. $X^\top X + \lambda \mathbb{I}$ invertible for $\lambda > 0 \Rightarrow$ unique solution; strictly shrinks $\|w\|$.
- Lasso** (L_1): $R = \|w\|_1$; subgradient $\lambda \text{sgn}(w_j)$. Sparsity: feature kept only if its MSE-gradient contribution $\geq \lambda$, else $w_j = 0$.
- Elastic net**: $L_1 + L_2$.

Effect: shrink weights $\rightarrow$ less variance, more bias.

2.6 Dual Models & Kernels

Representer theorem (informal; ridge, L_2 -logistic, SVM): optimal $w = \sum_{j=1}^n \alpha_j x^{(j)}$ (linear combination of training inputs) $\Rightarrow$ **Dual model**

$$h_\alpha(x) = \sum_j \alpha_j \langle x^{(j)}, x \rangle.$$

Primal: $d+1$ weights; dual: n - wins when transformed dim. $M \gg n$. (Ridge proof: push-through $(X^\top X + \lambda \mathbb{I})^{-1} X^\top = X^\top (X X^\top + \lambda \mathbb{I})^{-1}$.)

Kernel trick Replace $\langle \cdot, \cdot \rangle$ with **Kernel** $k_\phi(u, v) = \langle \phi(u), \phi(v) \rangle$ - never expand ϕ .

$$h_\alpha^\phi(x) = \sum_j \alpha_j k_\phi(x^{(j)}, x) \quad (\text{kernel machine}).$$

Common families:

- Linear** $u^\top v$.
- Polynomial** $k_{pk} = (u^\top v)^k$.
- RBF / Gaussian** $\exp(-\|u - v\|^2 / (2s^2))$ - **implicit infinite-dimensional ϕ** .

Mercer (informal): any continuous symmetric positive-definite kernel corresponds to some ϕ (so k can be designed directly).

Hyperplane classifier $y \in \{-1, +1\}$:

$$h_w(x) = \text{sign}(w^\top x), \quad \text{distance} = |w^\top x| / \|w\|.$$

Zero-offset plane $w^\top u = 0$, normal w .

3 Neural Networks

Idea: instead of handcrafting ϕ , jointly learn feature transform ϕ and predictor h end-to-end via a NN; compose into $h(\phi(x))$.

3.1 Neuron & Architectures

Neuron: $z = \sum_{j=0}^d w_j x_j$, $a = g(z)$ (with dummy $x_0 = 1$ so w_0 is bias). Linear regression = identity g ; logistic regression = sigmoid g ; both $\hat{y} = a$. A neuron is a hyperplane $\Rightarrow$ does NOT, AND, OR, but **not XOR (linearly inseparable)**. **NN** = computation graph $G = (V, E)$; node v holds h_v with parameters $w^{[v]}$; edges = data flow. **FFNN** = DAG (input $\rightarrow$ output, no cycles). **FCNN** = FF-NN of sequential **FC layers**, where every input node connects to every neuron of the layer (parallel neurons). **Depth** L = #layers (**excludes input**); **Width** = #neurons in a layer. **Deep learning**: ≥ 2 hidden layers. Containment: FCNN $\subset$ FFNN $\subset$ NN. **XOR**: not a single neuron; build via $AB + \bar{A}\bar{B}$ as a 2-layer net (NOTs $\rightarrow$ ANDs $\rightarrow$ OR). **Universal approximation** (informal): single hidden layer + non-linear g + sufficient width $\Rightarrow$ approximates any continuous function on bounded region. Depth helps in practice.

3.2 Activations

g usually non-linear (else network collapses to one linear map):

- Identity** $g(z) = z$ (regression output).
- Sigmoid** $\sigma(z) = 1 / (1 + e^{-z}) \in (0, 1)$.
- Tanh** $(e^z - e^{-z}) / (e^z + e^{-z}) \in (-1, 1)$.

- **ReLU** $\max(0, z)$.
- **Leaky ReLU** $\max(\alpha z, z)$, e.g. $\alpha=0.2$.
- **Softmax** (output for K -way classification) $\text{softmax}(z)_i = e^{z_i} / \sum_j e^{z_j} \in [0, 1]$, sums to 1.

Derivatives in miscMath.

3.3 FCNN Forward Pass

Per-layer dimensions: $W^{[l]} \in \mathbb{R}^{d_{l-1} \times d_l}$ (rows = sources, cols = destinations); entry $W_{ij}^{[l]}$ links source $i \in l-1$ to destination $j \in l$. With $a^{[0]} = x$, for $l=1:L$:

$$z^{[l]} = (W^{[l]})^\top a^{[l-1]}, \quad a^{[l]} = g^{[l]}(z^{[l]}), \quad \hat{y} = a^{[L]}.$$

Bias absorbed by augmenting $a^{[l-1]}$ with a leading 1 (or kept as separate $b^{[l]}$).

3.4 Tasks & Output Layers

General view: **Feature transform** ϕ (hidden layers) + **Predictor** h (output layer) jointly learned. Data $D = \{(x^{(i)}, y^{(i)})\}$, $x^{(i)} \in \mathbb{R}^d$.

- **Single regression** ($y \in \mathbb{R}$): 1 output neuron, identity g ;

$$J_{MSE} = \frac{1}{2n} \sum_i (\hat{y}^{(i)} - y^{(i)})^2.$$

- **Multi-output regression** ($y \in \mathbb{R}^m$): m identity neurons;

$$J_{MSE} = \frac{1}{2n} \sum_i \sum_{j=1}^m (\hat{y}_j^{(i)} - y_j^{(i)})^2.$$

- **Binary classification** ($y \in \{0, 1\}$): 1 sigmoid neuron; J_{BCE} in miscMath.
- **Multi-class (K): one-hot** $y \in \{0, 1\}^K$; K softmax neurons; **Categorical CE**

$$\text{CCE}(y, \hat{y}) = - \sum_{i=1}^K y_i \log \hat{y}_i, \quad J_{CCE} = \frac{1}{n} \sum_i \text{CCE}(\hat{y}^{(i)}, y^{(i)}).$$

- **Multi-label**: split outputs per group; sum task losses (e.g. softmax + CCE for one group, sigmoid + BCE for an independent binary label).

3.5 Backpropagation

Goal: compute $\partial J / \partial W^{[l]}$ for all l to run GD

$w_j \leftarrow w_j - \gamma \partial J / \partial w_j$ (γ learning rate). Backprop = DP application of chain rule (reverse-mode autodiff in libraries; in PyTorch: `loss.backward()`). Two passes:

Forward cache $z^{[l]}, a^{[l]}$ for $l=1:L$.

Backward let $\delta^{[l]} := \partial J / \partial z^{[l]} \in \mathbb{R}^{d_l}$.

- Output layer: $\delta^{[L]} = (\partial J / \partial a^{[L]}) \odot g^{[L]'}(z^{[L]})$.
- Recurrence ($l=L-1, \dots, 1$): $\delta^{[l]} = (W^{[l+1]})^\top \delta^{[l+1]} \odot g^{[l]'}(z^{[l]})$.
- Weight gradient: $\partial J / \partial W^{[l]} = a^{[l-1]} (\delta^{[l]})^\top \in \mathbb{R}^{d_{l-1} \times d_l}$.

$\odot$ = Hadamard. Output-layer shortcuts: BCE + sigmoid $\Rightarrow \delta^{[L]} = \hat{y} - y$; CCE + softmax $\Rightarrow \delta_j^{[L]} = \hat{y}_j - y_j$ (miscMath). Identity all layers + MSE: $\delta^{[L]} = \hat{y} - y$, then $\delta^{[l]} = W^{[l+1]} \delta^{[l+1]}$. **Naïving**: backprop = inner gradient computation; colloquially “backprop” sometimes means the whole training loop.

4 Deep Architectures

Specialised layers for grid (CNN) and sequence (RNN, attention) data; everything trained end-to-end with backprop + GD.

2.1 Convolutional Neural Network

CNN: stack of conv + pool + FC layers; extracts spatial features by sliding small kernels. Local connectivity + weight sharing $\Rightarrow$ far fewer parameters than FC over images.

Conv layer Input $I_H \times I_W \times C$. **Kernel/filter** $K \times K \times C$ (channels MUST match input). Each kernel: slides over input, dot product per location $\rightarrow$ 1 **feature map**. #kernels = #output channels. Kernel weights + 1 **bias per kernel (channel-wise, NOT pixel-wise)**. Hyperparameters: stride S , padding P .

$$O = 1 + \lfloor \frac{I+2P-K}{S} \rfloor \quad (\text{per spatial axis})$$

Output volume $O_H \times O_W \times C_{out}$. Parameter count per layer:

$$K \cdot K \cdot C_{in} \cdot C_{out} + C_{out}.$$

Pooling layer **Max-pool** / **Average-pool**: window size W , stride S . Reduces spatial size, keeps channels. **Zero trainable parameters** (deterministic op).

Architecture & receptive field Typical pipeline: [conv + ReLU + pool] $\times k \rightarrow$ flatten $\rightarrow$ FC $\rightarrow \hat{y}$. **Receptive field** of a deep neuron = region of input it depends on; grows with depth, kernel size, stride.

4.2 Recurrent Neural Network

RNN: NN layer designed for sequential data $x^{[1]}, \dots, x^{[T]}$; maintains hidden state $h^{[t]}$ summarising the past. **Same RNN layer applied at every step (shared weights, NOT distinct copies upon “unrolling”)**.

Vanilla recurrence

$$h^{[t]} = g((W^{[xh]})^\top x^{[t]} + (W^{[hh]})^\top h^{[t-1]} + b^{[h]})$$

$h^{[0]} = \mathbf{0}$ default. Per-step output (e.g. word tagging) $\hat{y}^{[t]} = \sigma((W^{[hy]})^\top h^{[t]})$ via an FC layer (also shared across t).

Variants

• **Bi-directional**: forward $\vec{h}^{[t]}$ and backward $\overleftarrow{h}^{[t]}$ RNNs; concat $h^{[t]} = [\vec{h}^{[t]}; \overleftarrow{h}^{[t]}]$.

- **Stacked**: $h^{[l,t]} = f^{[l]}(h^{[l-1,t]}, h^{[l,t-1]})$.

• **Encoder-decoder (seq2seq)**: encoder reads $x_{enc}^{[1:T_x]}$ producing $h_{enc}^{[T_x]}$; decoder (**separate weights!**) starts at $h_{dec}^{[0]} = h_{enc}^{[T_x]}$, takes $\langle \text{SOS} \rangle$ then its own previous output, emits $\hat{y}^{[1:T_y]}$ until $\langle \text{EOS} \rangle$.

- **Image captioning**: $h^{[0]}$ initialised by image-feature vector (CNN encoder output) instead of $\mathbf{0}$.

Training (BPTT) **Back-propagation through time**: unroll over T steps then backprop. Loss $J = \sum_t J^{[t]}$; gradient w.r.t. $W^{[hh]}$ accumulates contributions from all time steps via repeated $W^{[hh]}$ Jacobians $\Rightarrow$ **vanishing / exploding gradients** for long sequences.

4.3 Attention & Transformer

Motivation: seq2seq RNN bottlenecks all source info into a single $h_{enc}^{[T_x]}$; attention lets each output token query all source tokens directly; processes tokens in parallel (no recurrence).

Self-attention layer Tokens stacked column-wise $X = [x^{[1]} \dots x^{[T]}] \in \mathbb{R}^{d_v \times T}$. Linear projections (**layer-specific weights, NOT shared across attention layers**): $Q = W^q X$, $K = W^k X$, $V = W^v X$.

$$C = V \text{softmax}_{\text{col}}(K^\top Q / \sqrt{d_k}) \in \mathbb{R}^{d_v \times T}.$$

Column t of C is a convex combination of value vectors weighted by query- t 's similarity to all keys; $\sqrt{d_k}$ stops softmax saturating.

Variants

- **Self-attention** W^{qs} , W^{ks} , W^{vs} : each token attends to all (encoder).

- **Masked self-attention** W^{qm} , W^{km} , W^{vm} : future positions masked $-\infty$ before softmax (decoder; preserves causality during training).
- **Cross-attention** W^{qc} , W^{kc} , W^{vc} : Q from decoder, K, V from encoder output.

Positional encoding (attention has no order built-in): add to inputs at the bottom,

$$x^{[t]} \leftarrow x^{[t]} + PE^{[t]},$$

with $PE_{2k}^{[t]} = \sin(t/L^{2k/d})$, $PE_{2k+1}^{[t]} = \cos(t/L^{2k/d})$, $L \approx 10^4$.

Attention NN block Per-token:

- Self-attention produces $c^{[t]}$.
- Residual add $x^{[t]} + c^{[t]}$.
- **Same feed-forward NN** applied independently to each token vector.
- **Same FC** for output $\hat{y}^{[t]}$.

Encoder and decoder use separate FFNs with different weights. For language generation the decoder emits $z^{[t]}$, the **representation / embedding of the predicted token (NOT one-hot)**; fed back as input at step $t+1$ until $\langle \text{EOS} \rangle$.

5 Unsupervised Learning

Most data (text / image / video) unlabelled; human labels = expensive (time / money / expertise). **Unsupervised learning**: train on inputs only (no human labels). **Self-supervised learning (SSL)**: subset where supervision is generated automatically from the data itself.

5.1 Pretext Tasks

Pretext task: a fabricated task whose **pseudo-labels** are auto-generated; solving it forces the model to learn useful features. Examples:

- Rotation prediction.
- Contrastive learning.
- Image inpainting.
- Next-token prediction.

Rotation prediction For each image $x^{(i)}$, generate 4 rotated versions $x^{(i)[j]}$, $j \in \{0, 1, 2, 3\}$ (rotations $0/90/180/270^\circ$); pseudo-label $y^{(i)[j]}$ is the one-hot of j . Total $4n$ rotated images. Train CNN feature extractor + 4-way softmax head with CCE loss. Forces the model to learn object orientation, edges, contours.

5.2 Pre-training + Fine-tuning

Downstream task: target task of interest. **Pre-trained model usually cannot be applied directly** (task mismatch, e.g. 4-class rotation $\neq$ binary classification).

Build a new model

- **Feature extractor**: reuse pre-trained one, initialise with pre-trained weights (better than random).
- **Predictor**: new, task-specific head, random init.

Fine-tuning strategies Train on labelled $\{(x^{(i)}, y^{(i)})\}_{i=1}^n$ with task loss (BCE binary, CCE multi):

- Update every weight.
- Update only a subset (e.g. freeze early layers, train predictor + last block).
- Direct application: skip fine-tune, use features as is – no annotation needed but lower accuracy.

5.3 Image Retrieval & Contrastive Loss

Category-level retrieval: given query x^q , return top- k database images of same class. Pipeline: feature-extract $\rightarrow$ similarity match $\rightarrow$ sort.

$$\text{sim}(x, y) = \frac{x \cdot y}{\|x\| \|y\|} \in [-1, 1] \quad (\text{Cosine similarity})$$

1 same direction, -1 opposite.

Contrastive fine-tuning Form pairs from labelled data: **Positive pair** $(x^{(i)}, x^{(i+)})$ same class; **Negative pair** $(x^{(i)}, x^{(i-)})$ different. Extract $z = f(x)$. Exponential similarity ($\tau \approx 0.1$): $s^{(i+)} = e^{\text{sim}(z^{(i)}, z^{(i+)})/\tau}$, $s^{(i-)}$ analogously.

$$J^{(i)} = -\log \frac{s^{(i+)}}{s^{(i+)} + \sum_{i' \in A(i)} s^{(i'-)}}, \quad A(i) = \{k : y^{(k)} \neq y^{(i)}\}$$

5.4 Common Misconceptions

Attention NN

- **FFN / FC weights are SHARED across tokens within a layer** (same NN applied to every token).
- **Encoder vs decoder use SEPARATE FFNs**.
- **Each attention layer has its OWN W^q, W^k, W^v** (self W^{qs}, W^{ks}, W^{vs} ; masked-self $W^{qm}, \dots$; cross $W^{qc}, \dots$).
- **Decoder output $z^{[t]}$ is the embedding / representation of the predicted token (NOT one-hot)**.

RNN

- **Unrolling does NOT create distinct layers** – same RNN weights repeated across all t .
- **Encoder & decoder use separate RNN layers with different weights**.
- **$h^{[0]}$ need not be $\mathbf{0}$** : in image captioning use a CNN's image-feature vector; in seq2seq init $h_{dec}^{[0]} = h_{enc}^{[T_x]}$.

CNN

- **Bias is channel-wise, NOT pixel-wise**: one bias per output channel, shared over all spatial locations.
- **Pooling layers have ZERO trainable parameters** – fixed deterministic ops (max / avg).